패션 에이전트 아키텍처

앱은 HTTPS로 API Gateway 하나만 부릅니다. 백엔드는 Java · Spring Boot, agent가 Gemini로 할 일을 골라 기능별 MCP 서버를 부릅니다. 사진 분석은 우리가 학습한 모델을 서버에서 직접 돌려 호출 비용을 줄이고, 옷장 추천은 RAG로 합니다. 각 상자 아래 칩이 추천 기술 스택입니다.

① 사용자 폰

application
Flutter · iOS 우선

- 음성 듣기 (STT)
- 음성 안내 (TTS)
- 카메라 촬영
- 큰 버튼 2개 화면
- 얼굴 구도 안내 (기기 안)

`Flutter` `speech_to_text` `flutter_tts` `camera` `ML Kit 얼굴` `Firebase Auth SDK` `FCM`

사진은 폰에 저장하지 않음

→ HTTPS · REST · JWT

② 백엔드 · Java / Spring Boot

API Gateway
앱의 유일한 입구

- 경로별 라우팅 · JWT 검증 · 사용량 제한

`Spring Cloud Gateway` `Redis 사용량 제한` `Nginx/Caddy HTTPS`

account · 회원정보
로그인, 토큰, 이름·생년월일 등 (항목별 건너뛰기)
`Spring Security` `Firebase Admin 토큰 검증` `Apple · 구글 로그인` `Spring Data JPA`

agent
말 뜻 파악 · 도구 고르기 · 맞춤 추천 문장
`Spring AI ChatClient` `MCP 클라이언트` `Gemini 3.5 Flash` `ChatMemory` `구조화 출력`

DB
계정 · 회원정보 · 일정 · 옷장(옷 설명 JSON) · 지인 · 확인 요청 기록
`PostgreSQL` `JSONB` `pgvector (closet RAG)`

배포
`Docker Compose` `VM 한 대 (8GB+)` `.env 비밀 값`

⇢ MCP · Streamable HTTP

③ MCP 서버 · Spring AI MCP Server (@Tool)

mcp-weather
날씨, 날씨에 맞는 옷 두께·재질, 일교차 겉옷
`Open-Meteo` `RestClient` `Caffeine 캐시 30분`

mcp-schedule
일정 기록·알림, 장소 분위기에 맞는 옷
`카카오 로컬 API` `FCM` `@Scheduled`

mcp-clothes-state
얼룩·이염·찢어짐 위치는 우리가 학습한 모델, 종류·격식은 이미 학습된 모델
`YOLO 파인튜닝 (얼룩·찢어짐)` `FashionCLIP (종류·격식)` `ONNX Runtime Java` `색 이름 규칙` `애매할 때만 Gemini`

mcp-face-check
화장 대칭은 얼굴 점으로 계산, 이에 낀 음식물은 학습한 분류 모델
`ML Kit 얼굴 윤곽점 (앱)` `좌우 색·위치 비교 규칙` `치아 분류 모델 (ONNX)`

mcp-closet · RAG
내 옷 + 코디 지식을 검색해서, 가진 옷 안에서 근거를 붙여 추천
`Spring AI VectorStore` `pgvector` `multilingual-e5 임베딩 (ONNX)` `Gemini Flash 답변 생성`

mcp-helper
AI가 애매하면 가족·친구에게 사진 보내 묻고 답 읽어 주기
`문자 (NHN Cloud SMS·솔라피)` `답장용 웹 링크` `R2 임시 사진 24시간`

→ HTTPS

④ 외부 서비스

Firebase Auth
Apple · 구글 로그인, 계정 연결

Gemini API
gemini-3.5-flash · agent 판단, closet 답변 문장. 사진은 우리 모델이 애매할 때만

모델 학습 환경
Google Colab GPU · Roboflow 라벨링 · Ultralytics → ONNX로 내보내기

Open-Meteo
날씨 · 키 없음

카카오 로컬 API
장소 이름 → 종류(예식장·장례식장 등)

FCM · APNs
일정 당일 알림

문자 발송
가족·친구 폰 → 웹 링크로 답하기

Cloudflare R2
helper용 사진만 임시 보관, 자동 삭제

일반 호출 (HTTPS · JDBC) — MCP 호출 — 노란 상자 = 데모에 꼭 필요한 경로 — 비밀 값(Gemini 키, DB 비밀번호, 문자 키)은 서버 환경 변수에만, 앱·레포에는 넣지 않음

FASHION AGENT · 요청 흐름

"옷 검사해 줘" 한 번이 흐르는 길

오늘 면접이 있는 사용자가 펼쳐 놓은 옷을 비추는 경우입니다.

1

말하고 찍기

앱이 말을 글자로 바꾸고(STT), 펼쳐 놓은 옷을 한 번 찍습니다.

`application · speech_to_text · camera`

2

게이트웨이 통과

앱이 POST /outfit-check로 글자·사진·토큰을 보냅니다. 게이트웨이가 토큰과 사용량을 확인하고 agent로 넘깁니다.

`Spring Cloud Gateway · Redis`

3

할 일 고르기

agent가 Gemini에 도구 목록을 주고 고르게 합니다. 이번에는 weather, schedule, clothes_state를 부릅니다.

`agent · Spring AI · Gemini`

4

MCP 호출

세 MCP 서버가 오늘 날씨, 오늘 일정과 장소 종류, 옷 상태를 돌려줍니다. 옷 상태는 우리가 학습한 YOLO 모델이 얼룩 위치까지 찾습니다.

`Open-Meteo · 카카오 로컬 · YOLO · FashionCLIP`

5

애매하면 사람에게

우리 모델의 확신도가 낮으면 그 사진만 Gemini에 다시 묻고, 그래도 애매하면 "가족에게 물어볼까요?"라고 합니다.

`Gemini (폴백) · mcp-helper · 문자`

6

음성 안내

agent가 결과를 한국어 문장으로 정리하면 앱이 읽어 줍니다. "면접이라 슬랙스가 좋아요. 오른쪽 소매 끝에 얼룩이 있어요."

`agent → application · flutter_tts`

사진은 우리 모델이 먼저 본다

학습시킨 모델을 서버 CPU에서 돌려 사진 한 장당 호출 비용이 없습니다. 모델은 색·종류·얼룩 같은 본 것만 돌려주고, 추천에는 항상 이유를 붙입니다.

확실하지 않으면 단정하지 않기

확신도가 낮으면 Gemini에 한 번 더 묻고, 그래도 애매하면 손으로 확인하라고 하거나 가족에게 묻습니다. 옷장 추천은 RAG로 가진 옷 안에서만 합니다.

비밀 값과 사진은 밖으로 새지 않게

API 키는 서버 환경 변수에만. 사진은 저장하지 않고, helper로 보낼 때만 24시간 뒤 지워지는 곳에 둡니다.

레포별 추천 기술과 대안

구성요소	레포	추천 기술 스택	고른 이유	대안
application	`fashion-agent`	Flutter, speech_to_text, flutter_tts, camera, Firebase Auth SDK, FCM	이미 동작 중. 기기 음성은 무료이고 VoiceOver 목소리와 같음	Google Cloud STT/TTS, 네이버 CLOVA (실기기에서 인식이 부족할 때)
API Gateway	`fashion-agent-gateway`	Spring Cloud Gateway, Redis 사용량 제한	팀 스택과 같고 라우팅·JWT 필터·사용량 제한을 설정으로 처리	Nginx, 발표 전엔 게이트웨이 없이 agent에 Spring Security
account · 회원정보	`fashion-agent-gateway`	Spring Security, Firebase Admin(ID 토큰 검증), Spring Data JPA	Apple·구글 로그인과 기기 이전용 계정 연결을 Firebase가 맡고 서버는 확인만	Spring Security OAuth2 + 자체 JWT, Keycloak
agent	`fashion-agent-agent`	Spring AI ChatClient, MCP 클라이언트, Gemini 3.5 Flash, ChatMemory	MCP 서버와 같은 프레임워크, 서버 목록만 적으면 도구가 붙음	LangChain4j, Google Gen AI Java SDK 직접 호출
MCP 공통	`mcp-*`	Spring AI MCP Server (Streamable HTTP, @Tool)	메서드에 어노테이션만 붙이면 도구가 됨. agent와 설정 방식이 같음	MCP Java SDK 직접, TypeScript MCP SDK(Workers)
weather	`mcp-weather`	Open-Meteo, RestClient, Caffeine 캐시	키 없음, 앱에서 이미 검증, 체감 온도·일교차가 한 번에	기상청 단기예보 API, OpenWeatherMap
schedule	`mcp-schedule`	카카오 로컬 API, FCM, @Scheduled	국내 장소 카테고리(예식장·장례식장)가 정확, 알림은 무료	네이버 지역 검색 + 폰 로컬 알림, Google Places + Calendar
clothes_state	`mcp-clothes-state`	얼룩·찢어짐·보풀: YOLO 파인튜닝 / 종류·격식: FashionCLIP(학습 없이) / 색: 픽셀 평균 → 색 이름 규칙 / 실행: ONNX Runtime Java / 폴백: Gemini	호출 비용 없이 서버 CPU로 1초 안팎. 위치(박스)가 나와서 "오른쪽 소매 끝" 안내 가능. 데이터가 필요한 건 결함 탐지 하나	Python FastAPI 추론 서버(MCP는 Java 유지), 앱 안 Core ML·TFLite
face_check	`mcp-face-check`	앱 ML Kit 얼굴 윤곽점 → 서버 대칭 규칙(눈썹·입술 위치, 볼·입술 색 좌우 비교) + 입 부분 치아 이물질 분류(MobileNet 파인튜닝, ONNX)	대칭은 학습 없이 계산. 학습이 필요한 건 치아 분류 하나라 데이터가 적게 듦	MediaPipe Face Landmarker(점 478개, Python 서버), 전부 앱 안 TFLite
closet (RAG)	`mcp-closet`	Spring AI VectorStore + pgvector, 임베딩 multilingual-e5-small(ONNX, 서버에서 무료), 답변 Gemini Flash. 검색 대상: 내 옷 설명 + 코디 지식 문서(드레스코드·색 조합)	가진 옷 안에서만, 어떤 옷·규칙을 근거로 했는지 말하며 추천. 같은 옷 중복 찾기도 같은 벡터로	Gemini 임베딩 API, Qdrant·Chroma 벡터 DB, LangChain4j
모델 학습	`각 mcp 레포 training/`	Python, Ultralytics YOLO, Roboflow 라벨링, Google Colab GPU, ONNX 내보내기	학습은 Python이 표준, ONNX로 내보내면 Java에서 그대로 실행	학교·개인 GPU, AWS SageMaker
helper	`mcp-helper`	문자(NHN Cloud SMS 또는 솔라피), 답장용 웹 링크, Cloudflare R2	가족은 앱 설치 없이 링크로 답함, 카카오 심사 없이 바로 가능	카카오 알림톡(채널·템플릿 심사), 같은 앱 도우미 모드 + FCM
DB	(공용)	PostgreSQL + pgvector	회원·일정 데이터와 RAG 벡터를 한 DB에	MySQL + 별도 벡터 DB, Firestore
배포	(공용)	Docker Compose, VM 한 대, Nginx/Caddy HTTPS	서비스 전부를 파일 하나로 올리고 내림	Google Cloud Run, Kubernetes(발표 후)

학습 데이터: 공개 원단 결함 데이터(AITEX 등)는 공장 원단 사진이라 펼쳐 놓은 옷과 달라서, 우리 옷 사진을 직접 찍어 라벨링해야 합니다(결함 종류마다 수백 장 목표). 지금 배포된 Gemini 옷 분석 서버는 모델이 준비될 때까지 데모용 예비로 켜 둡니다.

FASHION AGENT · 시작 순서 (제안)

한 명이 레포 하나씩, 데모 경로부터

레포 9개를 동시에 시작하지 않고, "옷 검사해 줘"가 끝까지 도는 경로의 레포 3개를 먼저 한 명씩 맡습니다. 첫날 인터페이스를 같이 정해 두면 각자 따로 만들어도 마지막에 바로 붙습니다.

11/4 – 11/5

0. 약속 정하기

- 앱 ↔ 게이트웨이 API 모양
- MCP 도구 이름·입출력
- JWT에 담을 값 (uid)
- Docker Compose 뼈대, .env 목록
- 학습용 사진 찍기·라벨링 시작

11/6 – 11/12

1. 데모 경로

- 게이트웨이 + account
- agent + mcp-weather
- clothes-state 모델 1차 학습 + MCP 서버
- 앱 연결 주소를 게이트웨이로

11/13 – 11/17

2. 기능 넓히기

- mcp-schedule (음성 일정 등록)
- mcp-closet RAG (옷 설명 임베딩·검색)
- face-check 대칭 규칙 + 치아 분류
- 모델 2차 학습 (틀린 사진 추가)
- 얼룩 위치 왼쪽·오른쪽 기준

11/18 – 11/19

3. 리허설

- 실제 아이폰으로 처음부터 끝까지
- 예비 서버로 바꾸는 방법 확인
- 발표 시나리오 녹화 (네트워크 대비)

11/20 발표

담당 A

fashion-agent-gateway

라우팅, JWT 검증, 사용량 제한, Firebase 토큰 확인, 회원정보 저장. Docker Compose 뼈대도 같이.

담당 B

fashion-agent-agent (+ mcp-weather)

Spring AI로 Gemini 연결, MCP 클라이언트, 대화 기억, 결과를 한국어 안내 문장으로. weather는 작아서 함께.

담당 C

mcp-clothes-state

얼룩·찢어짐 데이터 라벨링과 YOLO 학습(Colab), ONNX로 내보내 Spring AI MCP에서 실행. 응답 모양은 지금 ClothesScan 그대로. 모델이 늦으면 Gemini 폴백으로 데모.

```
// 앱 → 게이트웨이 (예시, 첫날 확정)
POST /outfit-check   Authorization: Bearer <JWT>
{ "utterance": "옷 검사해 줘", "image": "<base64 JPEG>" }
→ { "speech": "면접이라 슬랙스가 좋아요. ...",
    "garments": [ ...지금 ClothesScan 모양... ],
    "needsHelper": false }
```

```
// agent → MCP 도구 (예시)
weather.today(lat, lon)          → { feelsLike, min, max, rainProb }
schedule.today(uid)              → [{ title, time, placeType }]
clothes_state.inspect(image)     → ClothesScan + source: model|gemini
closet.recommend(uid, situation) → [{ garmentId, reason }]
helper.ask(uid, imageRef, q)     → { requestId }
```